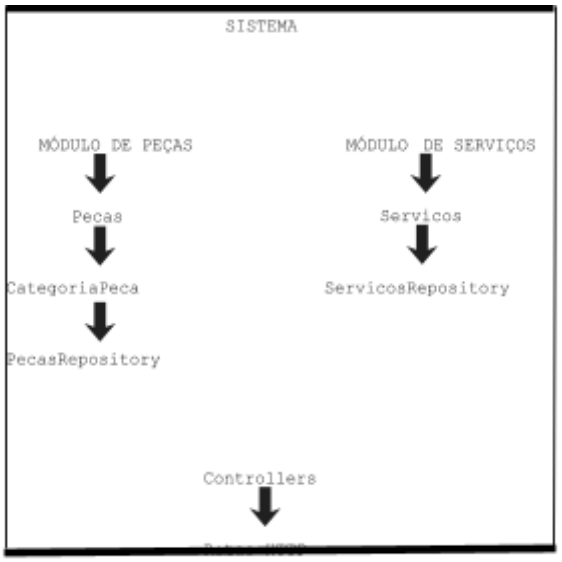
# Documentação

Estrutura dos Módulos, Repositórios e Rotas

## 8.1 Estrutura do sistema

Osistema foidesenvolvidocom uma divisão em dois módulos principais: **Peças** e **Serviços**. Cada módulo possui sua própria entidade de modelo, seu respectivo repositório e seu controller responsável pelo recebimento das requisições HTTP.




Cada módulo possui responsabilidades específicas. As entidades representam os dados manipulados pelo sistema, os repositories são responsáveis pelo armazenamento e pelas operações sobre esses dados, e os controllers realizam o mapeamento das requisições recebidas pela API.

## 8.2 Módulo de Peças

O módulo de**Peças** éresponsável pelo cadastro e gerenciamento das peças utilizadas no sistema.

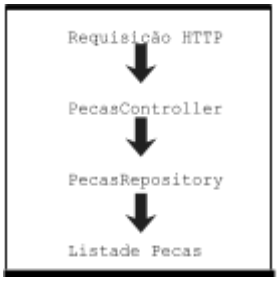
A entidade principal desse módulo é a classe **Pecas**, que representa uma peça e possui informações relacionadas ao seu cadastro, estoque, preços, identificação e categoria.

A classe possui atributos como:

- **codigoUnico**
- **nome**
- **fornecedorMarca**
- **quantidadeEstoque**
- **codigoDeBarras**
- **precoCusto**
- **precoVenda**
- **tamanho**
- **cor**
- **dataCadastro**
- **dataAtualizacao**
- **categoriaPeca**

O código único é gerado por meio de um contador **static**. Ele possui somente método getter, pois não deve ser alterado depois que a peça é criada.

A **dataCadastro** também não possui setter, pois deve representar o momento em que a peça foi criada. Já a **dataAtualizacao** é registrada quando ocorre uma alteração nos dados da peça.




## 8.3 Módulo de Serviços

O módulo de **Serviços** é responsável pelo cadastro e gerenciamento dos serviços disponibilizados pelo sistema.

A entidade principal é a classe **Servicos**, que representa um serviço.

Entre seus atributos estão:

• **codigoUnico**

• **nomeServico**

• **tempoEstimadoMinutos**

• **custoTabelado**

• **dataCriacao**

• **dataAtualizacao**

Assim como na entidade **Pecas**, o código único é gerado por meio de um contador static e não pode ser alterado posteriormente.

A data de criação é registrada automaticamente quando o objeto é criado, enquanto a data de atualização é registrada quando ocorre uma alteração nos dados do serviço.




## 8.4 Enum CategoriaPeca

O **CategoriaPeca** é um enum utilizado pela entidade **Pecas** para determinar a categoria de cada peça.




A utilização do enum permite que uma peça seja associada a uma categoria previamente definida no sistema, evitando valores diferentes ou inconsistentes para representar uma mesma categoria.

Dessa forma, o atributo **categoriaPeca** da entidade **Pecas** utiliza diretamente o tipo **CategoriaPeca**.

## 8.5 Repositórios

Os repositories são responsáveis pelo armazenamento e manipulação dos objetos durante a execução da aplicação.

Como o projeto não utiliza JPA, foram criadas listas para armazenar as peças e os serviços. Cada repository mantém sua própria lista.

Também foi utilizado o padrão **Singleton** nos repositories. O método **getInstance()** verifica se já existe uma instância do repository. Caso não exista, uma nova instância é criada; caso já exista, a instância existente é retornada.



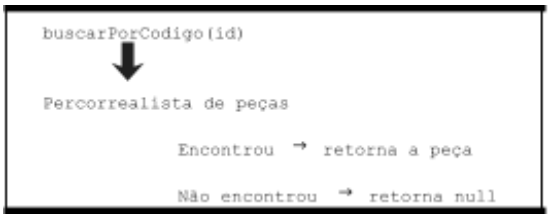

## 8.5.1 PecasRepository

O **PecasRepository** é responsável por armazenar e manipular a lista de objetos **Pecas**.

O repository possui uma lista de peças e utiliza o padrão Singleton para garantir que o controller tenha acesso à mesma instância do repository. **Principais funções**

### Buscar por código

Recebe o código único de uma peça e percorre a lista procurando uma peça com o mesmo código.




Essa função é utilizada para localizar uma peça específica e também serve de apoio para as operações de alteração e exclusão.

### Cadastrar




### Listar

Retorna todos os objetos existentes na lista de peças.

### Atualizar

Primeiro utiliza o método de busca pelo código. Caso a peça não seja encontrada, retorna null. Caso seja encontrada, seus dados permitidos são alterados e a data de atualização é registrada.

### Deletar

Também utiliza a busca pelo código. Caso a peça seja encontrada, ela é removida da lista e retornada. Caso não exista, retorna null.

## 8.5.2 ServicosRepository

O **ServicosRepository** possui funcionamento semelhante ao repository de peças, mas trabalha exclusivamente

com objetos **Servicos**.

Ele mantém uma lista de serviços e também utiliza o padrão Singleton por meio do método **getInstance()**.

### Principais funções

### Buscar por código




### Cadastrar serviço
Adiciona o serviço recebido à lista de serviços.

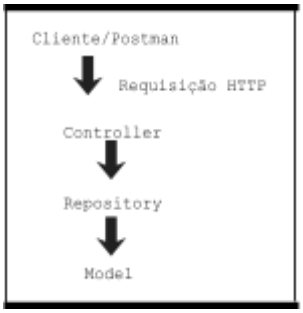
### Listar serviço
Retorna todos os serviços cadastrados no repository.

### Atualizar serviço
Procura o serviço pelo código. Se encontrado, altera os dados permitidos e registra a data de atualização.

### Deletar serviço
Procura o serviço pelo código. Caso exista, remove o objeto da lista e retorna o serviço removido.

## 8.6 Sistema de Rotas
O sistema de rotas é responsável por disponibilizar as funcionalidades dos módulos por meio de requisições HTTP.

Os controllers recebem as requisições, identificam o tipo da operação e chamam o método correspondente no repository.




As requisições utilizadas no sistema são:

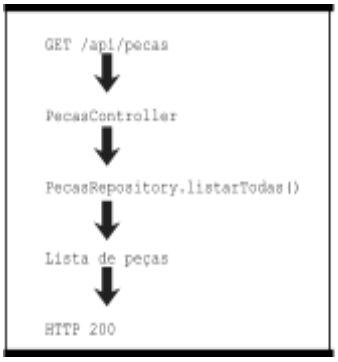
- **GET** — consultar/listar dados;
- **POST** — cadastrar dados;
- **PUT** — alterar dados;
- **DELETE** — excluir dados.

## 8.6.1 Rotas de Peças
O controller de peças utiliza a rota base **/api/pecas** e inicializa o **PecasRepository** utilizando seu método **getInstance()**.

### GET — Listar peças
**Rota:** GET /api/pecas

**Função:** retorna todas as peças cadastradas.

**Resultado esperado:** HTTP 200 OK.

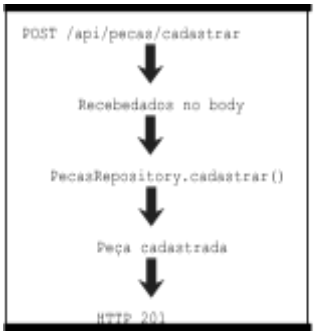



## POST — Cadastrar peça

**Rota:** POST /api/pecas/cadastrar

**Função:** recebe os atributos da peça no corpo (body) da requisição e realiza o cadastro.

**Resultado esperado:** HTTP 201 CREATED.

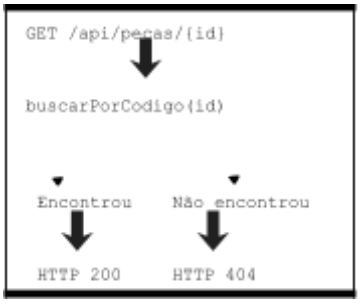



## GET — Buscar peça por código

**Rota:** GET /api/pecas/{id}

**Função:** procura uma peça utilizando seu código único.

Se a peça não for encontrada, o sistema retorna HTTP 404 NOT FOUND. Caso seja encontrada, retorna a peça com HTTP 200 OK.



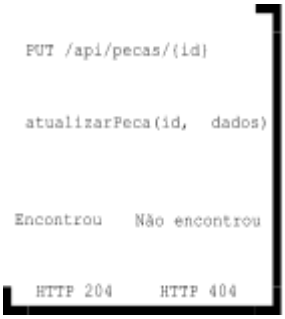

## PUT — Alterar peça

**Rota:** PUT /api/pecas/{id}

**Função:** altera os dados de uma peça utilizando seu código único.

O id é informado na rota e os atributos que serão alterados são enviados no corpo da requisição.

Caso a peça não seja encontrada, retorna HTTP 404 NOT FOUND. Caso seja encontrada, seus dados são alterados e a operação retorna HTTP 204 NO CONTENT.



### DELETE — Excluir peça

**Rota:** DELETE /api/pecas/{id}

**Função:** exclui uma peça utilizando seu código único.

Se a peça não existir, o sistema retorna HTTP 404 NOT FOUND. Caso seja encontrada, a peça é removida da lista e o sistema retorna HTTP 204 NO CONTENT.




## 8.6.2 Rotas de Serviços

O controller de serviços utiliza a rota base **/api/servicos** e inicializa o **ServicosRepository** para realizar as operações relacionadas aos serviços.

### GET — Listar serviços

**Rota:** GET /api/servicos

**Função:** retorna todos os serviços cadastrados.

**Resultado esperado:** HTTP 200 OK.




### POST — Cadastrar serviço

**Rota:** POST /api/servicos/cadastrar

**Função:** recebe os atributos do serviço no corpo da requisição e realiza seu cadastro.

**Resultado esperado:** HTTP 201 CREATED.




### PUT — Alterar serviço

**Rota:** PUT /api/servicos/{id}

**Função:** altera os dados de um serviço utilizando seu código único.

O id é informado na rota e os novos dados são enviados no corpo da requisição.

Caso o serviço não seja encontrado, retorna HTTP 404 NOT FOUND. Caso seja encontrado, os dados são alterados e a operação retorna HTTP 204 NO CONTENT.

```
PUT/api/servicos/{id}

   atualizarServico(id, dados)

    Encontrou Nãoencontrou

    HTTP 204     HTTP404
```

### DELETE — Excluir serviço

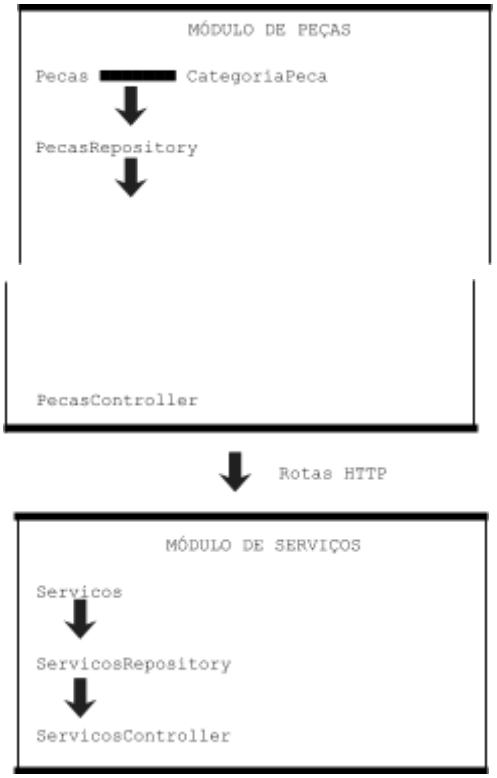
**Rota:** DELETE /api/servicos/{id}

**Função:** exclui um serviço utilizando seu código único.

Caso o serviço não seja encontrado, retorna HTTP 404 NOT FOUND. Caso seja encontrado, ele é removido da lista e o sistema retorna HTTP 204 NO CONTENT.




## 8.7 Resumo da arquitetura




Dessa maneira, cada parte do sistema possui uma responsabilidade definida. As entidades **Pecas** e **Servicos** representam os dados do sistema; o **CategoriaPeca** define as categorias possíveis para as peças; os repositories realizam o armazenamento e as operações de manipulação dos objetos; e os controllers disponibilizam essas funcionalidades por meio das rotas HTTP.

A utilização do padrão Singleton nos repositories permite que os controllers trabalhem com uma única instância de cada repository durante a execução da aplicação, mantendo as listas utilizadas para o armazenamento em memória.